A Report on

Intrusion Detection System Using Smart Contracts

carried out as part of the course Minor Project CS3270

Submitted by

Aashish Singh
Roll No: 229301585
VI-CSE-J

Under the Guidance of:

Dr. Prashant Vats

Manipal University Jaipur

Department of Computer Science and Engineering

February 2025

Certificate

This is to certify that the project entitled **Intrusion Detection System Using Smart Contracts** is a bonafide work carried out as Minor Project Midterm Assessment (Course Code: CS3270) in partial fulfillment of the award of the Bachelor of Technology degree in Computer Science and Engineering by **Aashish Singh** bearing registration number **229301585**, during the academic semester VI of the year 2024-2025.

Project Guide: Dr. Prashant Vats

Signature: ______________

Acknowledgement

I would like to express my sincere gratitude to my internal supervisor **Dr. Prashant Vats** for his constant support, guidance, and continuous engagement. I am also grateful to Dr. Neha Chaudhary, Head of the Department of CSE, for her valuable guidance and encouragement.

Abstract

SmartLock is a next-generation access control system leveraging blockchain technology to provide secure, immutable, and transparent locking solutions. This project aims to integrate blockchain-based intrusion detection mechanisms to enhance security by detecting unauthorized access attempts and automatically triggering preventive measures. The system uses smart contracts to ensure that all operations are logged immutably on a blockchain, providing a robust audit trail.

Contents

1 Introduction **6**
1.1 Objective of the Project . . . 6
1.2 Motivation and Background . . . 6
1.3 Problem Statement . . . 6
1.4 Scope of the Project . . . 6
1.5 Technology Used . . . 6
1.5.1 Hardware Requirements . . . 6
1.5.2 Software Requirements . . . 7

2 Literature Review **8**
2.1 Blockchain in Security . . . 8
2.2 Intrusion Detection Systems (IDS) . . . 8
2.3 Smart Contracts for Access Control . . . 8

3 Design Description **9**
3.1 System Architecture . . . 9
3.2 Flowchart . . . 10
3.3 Data Flow Diagram . . . 11
3.4 Module Description . . . 12
3.4.1 User Interface . . . 12
3.4.2 Backend Server . . . 12
3.4.3 Smart Contract Module . . . 12
3.4.4 Database . . . 12

4 Project Description **13**
4.1 Implementation Details . . . 13
4.2 Smart Contract Implementation . . . 13
4.3 Integration . . . 13
4.4 Challenges Faced . . . 13

5 Testing and Implementation **14**
5.1 Testing Methodology . . . 14
5.2 Test Cases and Results . . . 14
5.3 Performance Evaluation . . . 14

6 Conclusion and Future Work **15**
6.1 Conclusion . . . 15
6.2 Limitations . . . 15
6.3 Future Work . . . 15

6.4 Impact and Recommendations 15

Chapter 1

Introduction

1.1 Objective of the Project

The objective of this project is to develop an intrusion detection system using smart contracts to ensure security, immutability, and transparency.

1.2 Motivation and Background

In today's digital era, security breaches and unauthorized access can lead to significant financial and data losses. With blockchain's decentralized and tamper-proof nature, integrating it with traditional security systems offers a promising approach. This project is motivated by the need to create a system that not only controls access but also provides an immutable log of all events.

1.3 Problem Statement

Traditional access control systems often lack transparency and are vulnerable to tampering. The challenge is to design a system that combines the benefits of blockchain technology with conventional security measures to detect and prevent intrusions effectively.

1.4 Scope of the Project

This report covers the design, implementation, testing, and evaluation of a blockchain-based intrusion detection system. The project integrates web technologies (Node.js, React.js, MongoDB) with Ethereum-based smart contracts to secure access and record all events on a blockchain ledger.

1.5 Technology Used

1.5.1 Hardware Requirements

- Computer with internet access.
- Server for backend hosting.

1.5.2 Software Requirements

- Node.js, React.js, MongoDB.
- Ethereum blockchain, Solidity, Hardhat.

Chapter 2

Literature Review

2.1 Blockchain in Security

Blockchain technology has been increasingly adopted for security applications due to its inherent features like decentralization, transparency, and immutability. Studies have demonstrated its effectiveness in preventing data tampering and ensuring trust.

2.2 Intrusion Detection Systems (IDS)

IDS have traditionally used signature-based and anomaly-based methods. Recent research indicates that integrating blockchain can enhance these systems by ensuring a secure and tamper-proof log of events.

2.3 Smart Contracts for Access Control

Smart contracts allow for automated, trustless transactions and are ideal for enforcing access control policies. Their ability to log events immutably makes them a valuable component in security systems.

Chapter 3

Design Description

3.1 System Architecture

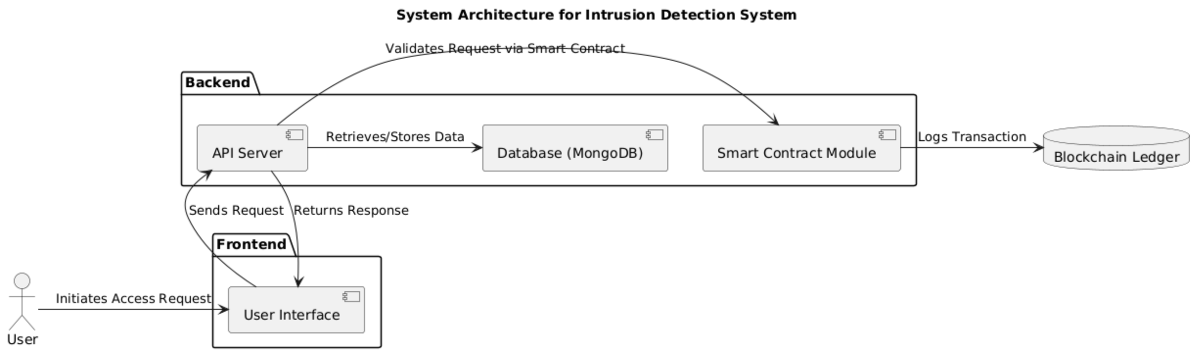

Figure 3.1: System Architecture of the Intrusion Detection System

3.2 Flowchart

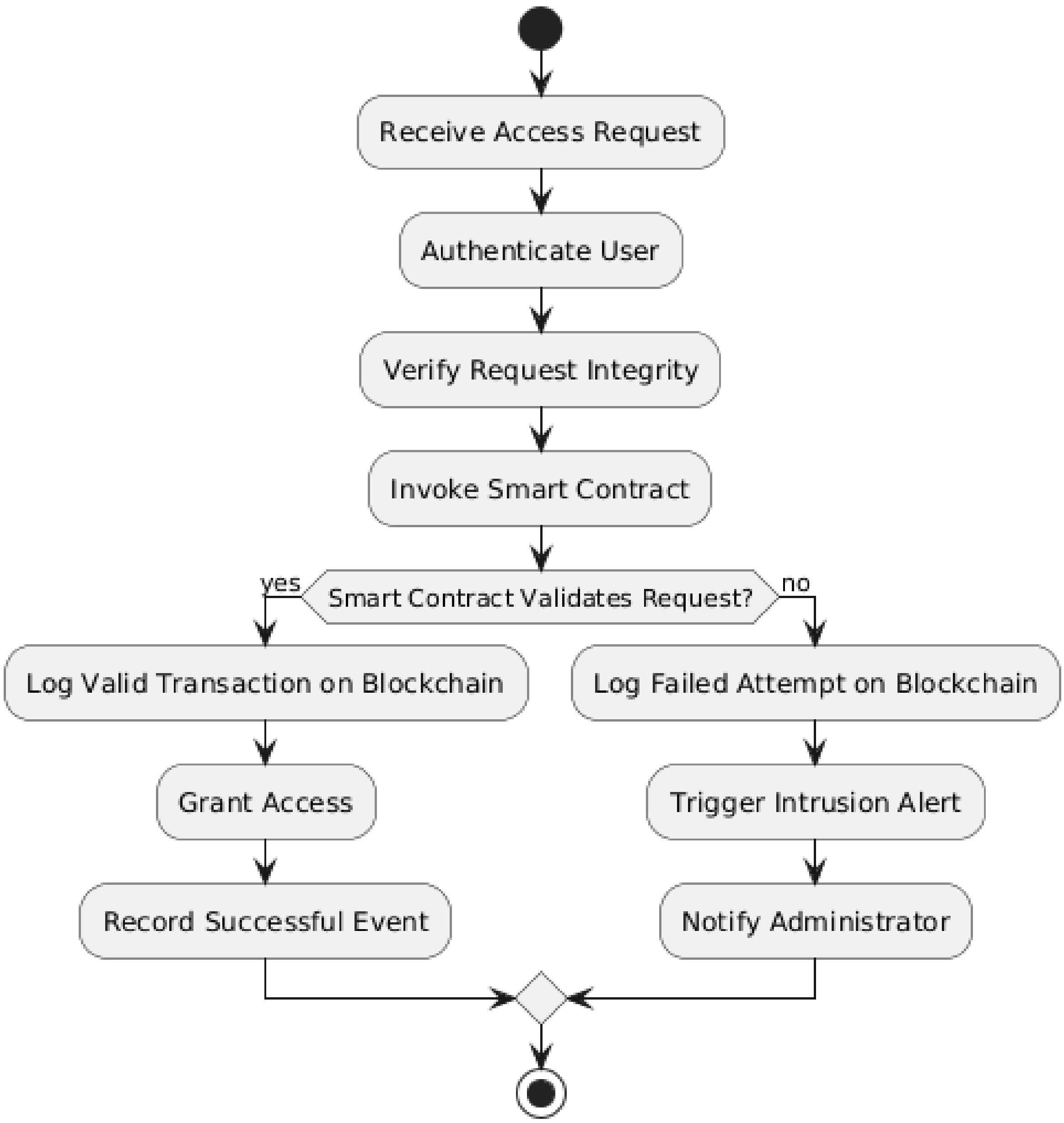

Figure 3.2: Flowchart of the System

3.3 Data Flow Diagram

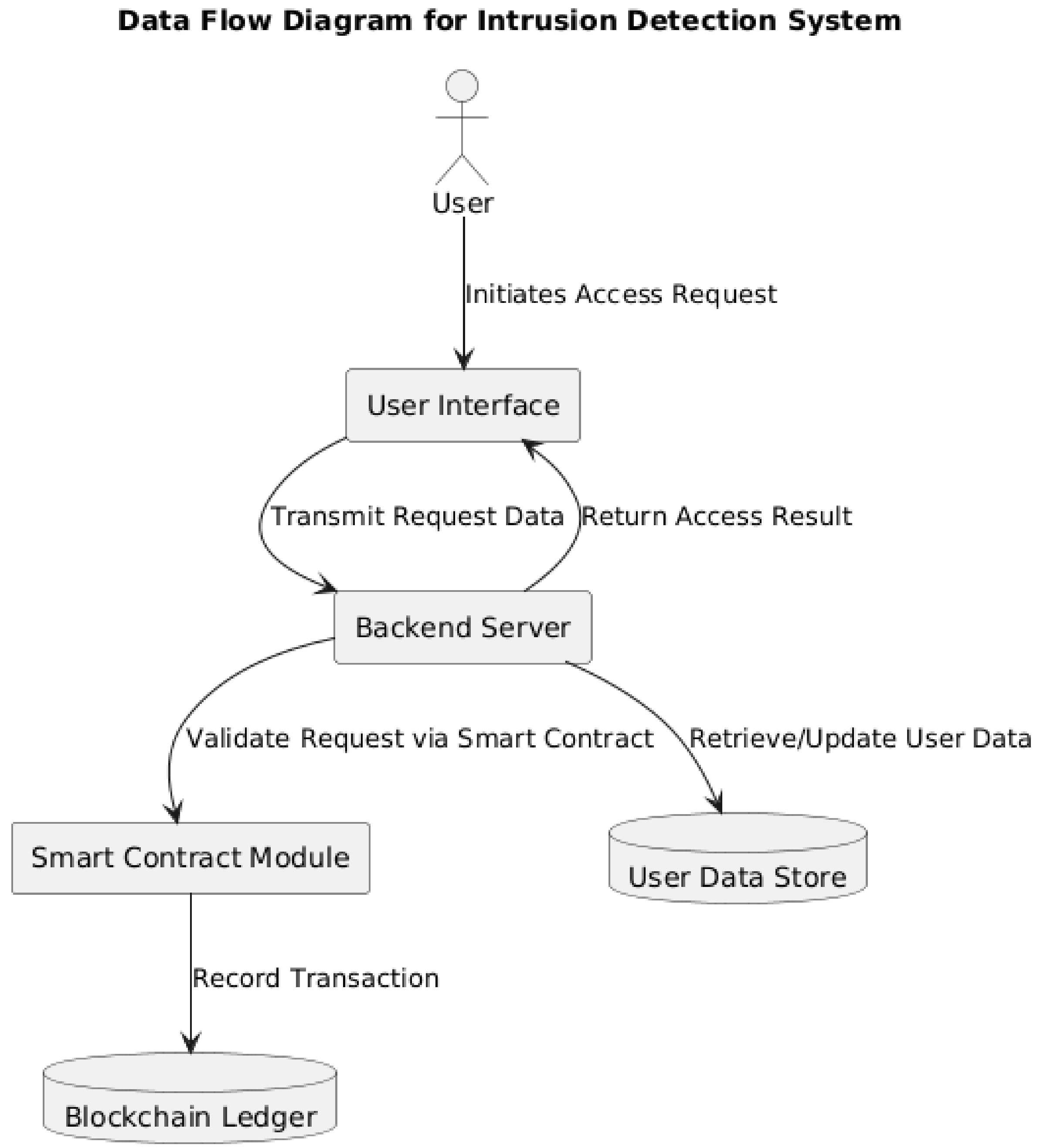

Figure 3.3: Data Flow Diagram of the System

3.4 Module Description

3.4.1 User Interface

Developed using React.js, the user interface provides a responsive environment for both administrators and end-users, ensuring ease of use and intuitive navigation.

3.4.2 Backend Server

The backend, built on Node.js and Express.js, handles data processing, authentication, and communication with the smart contract module.

3.4.3 Smart Contract Module

Smart contracts, written in Solidity and deployed on an Ethereum test network, manage access permissions and log every transaction in an immutable manner.

3.4.4 Database

MongoDB is used to store ancillary data such as user information and system logs, facilitating fast data retrieval and processing.

Chapter 4

Project Description

4.1 Implementation Details

The project is implemented in three major parts: the frontend, the backend, and the smart contracts. The frontend interacts with users and communicates with the backend via RESTful APIs. The backend processes these requests, interacts with the blockchain, and stores necessary data in the database.

4.2 Smart Contract Implementation

The smart contracts are developed in Solidity and handle tasks such as:

- Managing user access permissions.
- Recording each access attempt (successful or unsuccessful).
- Triggering alerts for suspicious activities.

The contracts are rigorously tested using the Hardhat framework to ensure security and reliability.

4.3 Integration

All components are integrated through well-defined APIs. The frontend sends access requests to the backend, which then validates these requests by invoking the smart contracts and updating the blockchain accordingly.

4.4 Challenges Faced

The project encountered several challenges, including:

- Integrating blockchain with conventional web technologies.
- Ensuring the scalability of the system.
- Addressing potential vulnerabilities in smart contract code.

These challenges were mitigated through iterative testing and by following industry best practices.

Chapter 5

Testing and Implementation

5.1 Testing Methodology

The system underwent comprehensive testing, which included:

- **Unit Testing:** Testing individual modules.
- **Integration Testing:** Ensuring seamless interaction between the frontend, backend, and blockchain components.
- **Security Testing:** Conducted to identify and address vulnerabilities in the smart contracts.

5.2 Test Cases and Results

Various test cases were simulated to mimic real-world scenarios such as:

- Valid access requests.
- Unauthorized access attempts.
- System overload and recovery.

The results confirmed that the system reliably detected unauthorized access and logged all events on the blockchain.

5.3 Performance Evaluation

Performance metrics such as response time and throughput were evaluated. Although blockchain integration introduced minor delays, the overall performance was satisfactory given the enhanced security and auditability.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project successfully demonstrates the integration of blockchain technology and smart contracts into a traditional intrusion detection system. The resulting system not only enhances security through immutable logging but also provides a transparent audit trail for all access events.

6.2 Limitations

Despite its advantages, the system has certain limitations:

- Scalability challenges due to blockchain transaction latency.
- Increased complexity in smart contract debugging and maintenance.

6.3 Future Work

Future improvements can include:

- Integrating AI-based anomaly detection to further enhance threat identification.
- Optimizing smart contract performance for faster transaction processing.
- Expanding support for additional authentication mechanisms.

6.4 Impact and Recommendations

The project highlights the potential of blockchain in creating secure and transparent systems. It is recommended that future developments consider a hybrid approach, combining traditional security measures with blockchain technology to mitigate scalability issues.

Bibliography

[1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*

[2] Ethereum Foundation. (2024). *Ethereum Developer Documentation.*

[3] Buterin, V. (2014). *A Next-Generation Smart Contract and Decentralized Application Platform.*